

Consensus Algorithms

Phase 3 · Advanced · Estimated time: 2 hours

1. Concept From First Principles

Chapter 1 established that nodes can't always tell whether another node is alive, and that clocks can't be trusted for ordering. This raises an uncomfortable question that Phase 2's replication chapter quietly sidestepped: when a single-leader database's leader dies, *how do the remaining nodes agree on which follower becomes the new leader*, without any node being able to fully trust its own view of who's alive? This is the **consensus problem**, and it is one of the hardest, most rigorously studied problems in all of distributed computing.

Formally, a consensus algorithm must guarantee three properties despite node failures and network partitions: **agreement** (every non-faulty node that decides, decides on the same value), **validity** (the decided value was actually proposed by some node, not invented), and **termination** (every non-faulty node eventually decides something — the system doesn't hang forever). Achieving all three simultaneously, correctly, in the presence of the partial failures and clock skew from Chapter 1, turns out to be surprisingly subtle — naive approaches ("just have the node with the lowest ID declare itself leader") break in obvious ways once you consider a partition where two nodes each believe they're isolated and both promote themselves.

Backend engineer analogy

Think of a team of five on-call engineers who need to agree on who's the incident commander for tonight, but their Slack connection is flaky and messages sometimes arrive late or out of order. 'Whoever posts first' breaks if two people post at nearly the same time and neither sees the other's message in time. Consensus algorithms are the rigorously-proven version of the coordination protocol needed to make this work correctly even under exactly that kind of message flakiness — which is why they're notoriously hard to get right by intuition alone.

Paxos: correct but famously hard to understand

Paxos, published by Leslie Lamport in 1998, was the first widely-accepted, formally proven-correct consensus algorithm, and remains foundational — but it earned a well-deserved reputation for being extremely difficult to understand and implement correctly, even for experienced distributed-systems engineers. At a conceptual level (the depth this program needs, not full proof-level detail), Paxos works through rounds of **proposals**: a node proposes a value with a unique, increasing proposal number; other nodes either promise to accept it (if they haven't already promised something with a higher number) or reject it; once a proposer gets acknowledgment from a majority, the value is considered chosen. Paxos notably doesn't define an explicit, ongoing leader role in its base form, which is part of why real systems built on it (like Google's Chubby lock service) needed substantial additional engineering to make practical.

Raft: designed for understandability

Raft, published in 2014 by Diego Ongaro and John Ousterhout specifically to be more understandable than Paxos while providing equivalent guarantees, is now the dominant consensus algorithm in real-world systems (etcd, Consul, and many others use it directly). Raft makes leadership explicit and central: at any time, each node is a **follower**, **candidate**, or **leader**. Followers passively receive log entries and periodic heartbeats from the leader. If a follower doesn't receive a heartbeat within a randomized **election timeout** (commonly 150–300ms, randomized specifically to reduce the chance of two nodes starting elections simultaneously), it becomes a candidate, increments a term number, votes for itself, and requests votes from other nodes. The first candidate to receive votes from a **majority** of nodes becomes the new leader for that term. All writes then flow through this leader, which replicates its log to followers — a structure that should feel familiar from Phase 2's single-leader replication chapter, because Raft is essentially the rigorously-correct algorithm for managing exactly that leader-follower relationship, including safely handling leader failure and re-election.

The **majority (quorum)** requirement is the load-bearing safety mechanism in both algorithms: because any two majorities out of the same set of nodes must overlap by at least one node, it's mathematically impossible for two different candidates to both win an election in the same term, and impossible for two different values to both be "chosen" for the same log position. This is exactly why consensus systems are typically deployed with an **odd number** of nodes (3, 5, 7) — a 5-node cluster tolerates 2 node failures while still being able to form a majority (3) to keep operating, whereas a 4-node cluster only tolerates 1 failure for the same reason a majority of 4 requires 3, no better than a 3-node cluster's requirement of 2, while costing more hardware.

It's worth being explicit about where consensus algorithms are and aren't the right tool, since this is a common interview trap. Consensus is expensive — every decision requires a network round-trip to a majority of nodes, adding real latency, which is precisely why you don't run every database write through a full consensus protocol. In practice, consensus algorithms are used for a relatively small amount of critical *coordination* metadata (who is the current leader, what is the current cluster configuration, who holds a distributed lock) rather than for high-volume application data itself — which is exactly the role Chapter 3's coordination services (Zookeeper, etcd) play, sitting on top of a consensus algorithm internally so that the rest of your system doesn't have to implement Raft or Paxos itself.

2. Real-World Example / Case Study

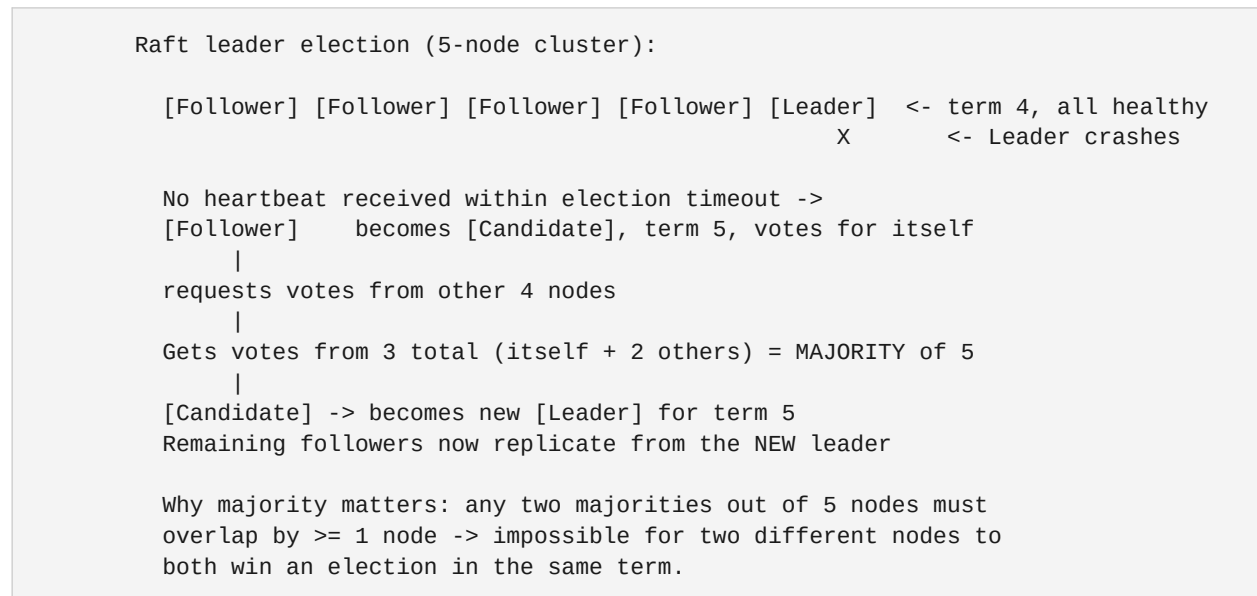
Kubernetes, which underpins service discovery and orchestration at Careem-, Talabat-, and Noon-scale engineering organizations (Phase 2 Chapter 12), uses etcd — a Raft-based distributed key-value store — to store all cluster state, including which pods are running where. Etcd's Raft consensus is what guarantees that even if some etcd nodes crash, the cluster's view of "what should be running" stays consistent and doesn't fork into two conflicting versions.

Distributed databases like CockroachDB and TiDB, increasingly used by fintechs needing strong consistency with horizontal scale (relevant to Razorpay-, PhonePe-, or Tabby-style ledger systems), use Raft internally per data shard/range to manage leader election and replicated writes — giving them single-leader-replication-like simplicity per shard while surviving individual node failures automatically, without a human manually promoting a new leader the way older systems required.

Apache Kafka's newer KRaft mode (replacing its earlier Zookeeper-based coordination, foreshadowing Chapter 3) uses a Raft-based consensus protocol to manage which broker is the controller for cluster

metadata, a direct real-world illustration of a message-queue system (Phase 2 Chapter 8) depending on consensus underneath its own coordination layer.

3. Diagram



4. Trade-offs Table

Aspect	Paxos	Raft
Understandability	Notoriously difficult	Explicitly designed to be understandable
Leadership model	No explicit leader in base form	Explicit, single leader per term
Real-world adoption	Used historically (Chubby), less directly today	Dominant in modern systems (etcd, Consul)
Correctness guarantee	Proven correct	Proven equivalent to Paxos's guarantees
Typical use today	Foundational theory, some legacy systems	Default choice for new coordination systems

5. Common Interview Questions

Q1: "Why do you need consensus at all — why can't a node just decide things on its own?"

Model approach: Explain that in a partitioned or partially-failed system, a single node cannot reliably know the true global state, so unilateral decisions (like declaring yourself the leader) can produce two conflicting decisions (split-brain, Phase 2 Ch 5) unless a majority-based agreement protocol is used.

Q2: "Explain Raft's leader election process at a high level."

Model approach: Walk through the follower -> candidate -> leader state machine, the randomized election timeout, the term-numbered vote request, and the majority-vote requirement to become leader,

tying it back to why randomization reduces split-vote elections.

Q3: "Why do consensus clusters typically use an odd number of nodes?"

Model approach: Show the majority math concretely — a 5-node cluster needs 3 for a majority and tolerates 2 failures; a 4-node cluster still needs 3 for a majority but only tolerates 1 failure, so it pays for a 4th node without gaining fault tolerance over a 3-node cluster.

Q4: "Would you run every database write through a Raft consensus protocol? Why or why not?"

Model approach: No — explain that consensus adds real per-decision latency (a network round-trip to a majority), so it's reserved for critical coordination metadata (leader identity, cluster config, locks) rather than high-volume application data, which is why dedicated coordination services exist as a layer on top (Chapter 3).

6. Practice Exercises

- 1 Trace through a Raft election in a 3-node cluster where the leader crashes, including what happens if two followers' election timeouts fire at nearly the same time.
- 2 Explain why a 6-node cluster doesn't tolerate more failures than a 5-node cluster, using the majority-quorum math from this chapter.
- 3 Describe, in your own words, why Paxos is considered harder to understand than Raft, referencing the explicit-leader design difference.

7. Curated YouTube Videos

Language	Video & Channel	Why it helps
English	"Consensus Algorithms Explained: Raft, Paxos & PBFT for Beginners"	Beginner-friendly comparison of the major consensus algorithms.
English	"Raft Explained – A Leader-Based Consensus Algorithm"	Focused, diagram-driven walkthrough of Raft's mechanics.
Hindi	"System Design Complete Course 2026" (Hindi + English, 10+ hrs)	Covers consensus and leader election as part of a full bilingual course.
Hindi	"Complete System Design Roadmap 2026 HLD & LLD" — CodeHelp	Places consensus algorithms in context of the broader Hindi HLD roadmap.

8. Key Takeaways

- Consensus algorithms let a group of nodes agree on a value despite failures and partitions, guaranteeing agreement, validity, and termination.
- Paxos was the first proven-correct consensus algorithm but is notoriously hard to understand and implement.
- Raft was explicitly designed for understandability and is now the dominant real-world choice (etcd, Consul).

- Raft's follower/candidate/leader state machine with randomized election timeouts elects a single leader per term.
- Majority (quorum) requirements make split-brain elections mathematically impossible within a term.
- Odd-numbered clusters (3, 5, 7) give better fault tolerance per node than even-numbered ones.
- Consensus is reserved for critical coordination metadata, not high-volume application writes, due to its latency cost.